

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЛАБОРАТОРНА РОБОТА №1

з дисципліни «Об'єктно-орієнтоване програмування»

на тему

**«Знайомство із середовищем розробки програм Microsoft Visual Studio
та складання модульних проєктів програм на C++»**

Виконав:

Чакун	Олександр	Сергійович
студент	групи	ІМ-о51
номер	у	списку: 16

Перевірив:

Рекечинський Дмитро Олександрович

Київ - 2026

1. Мета та завдання роботи

Мета роботи - ознайомитися зі створенням Win32-застосунків у Microsoft Visual Studio та закріпити роботу з віконними повідомленнями, ресурсами, модальними діалогами, елементами керування і роздільною компіляцією модулів C++.

Завдання - створити C++-проект, реалізувати два пункти меню відповідно до індивідуальних варіантів, винести кожну роботу в окремий незалежний модуль, перевірити виконання програми та оформити результати.

2. Індивідуальний варіант

Пункт	Розрахунок	Реалізація
Робота 1	$B1 = 16 \bmod 4 = 0$	Edit Control, кнопки «Так»/«Відміна», виведення введеного тексту.
Робота 2	$B2 = (16 + 1) \bmod 4 = 1$	Горизонтальний Scroll Bar 1-100, виведення вибраного числа.

3. Структура проєкту

Lab1/	src/	Lab1.cpp,	resource.h,	module1.h/.cpp,	module2.h/.cpp
	res/	Lab1.rc,	module1.rc,	module2.rc	

Головний модуль відповідає за створення вікна, меню і виведення результатів. module1 та module2 не включають один одного. Кожен модуль має лише одну зовнішню інтерфейсну функцію, а його діалогова процедура оголошена як static.

4. Реалізація головного вікна

Точка входу `wWinMain` реєструє клас головного вікна, підключає меню з ресурсів, створює вікно та запускає стандартний цикл обробки повідомлень.

```
int APIENTRY wWinMain(HINSTANCE hInstance, LPWSTR, int nCmdShow)
{
    const wchar_t CLASS_NAME[] = L"Lab1WindowClass";
    WNDCLASSEX wc{};
    wc.cbSize = sizeof(WNDCLASSEX);
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);
    wc.lpszMenuName = MAKEINTRESOURCEW(IDR_MAINMENU);
    wc.lpszClassName = CLASS_NAME;

    if (!RegisterClassExW(&wc)) return 0;

    HWND hwnd = CreateWindowExW(
        0, CLASS_NAME,
        L"Лабораторна робота №1 - Чакун Олександр Сергійович",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT,
        800, 500, nullptr, nullptr, hInstance, nullptr);

    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    MSG msg{};
    while (GetMessageW(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    return static_cast<int>(msg.wParam);
}
```

Адреса Callback-функції `WndProc` записується у поле `lpfnWndProc`. Меню завантажується з ресурсу `IDR_MAINMENU`. Функції `GetMessageW`, `TranslateMessage` та `DispatchMessageW` формують цикл повідомлень Windows.

5. Обробка команд і відображення результатів

У WndProc повідомлення WM_COMMAND обробляє команди меню, а WM_PAINT перемальовує клієнтську область головного вікна.

```
case
    switch (LOWORD(wParam))
    {
        case IDM_WORK1:
            if (Func_MOD1(hWnd, g_text, _countof(g_text)))
                InvalidateRect(hWnd, nullptr, TRUE);
            return 0;

        case IDM_WORK2:
            if (Func_MOD2(hWnd, &g_number))
                InvalidateRect(hWnd, nullptr, TRUE);
            return 0;

        case IDM_ABOUT:
            MessageBox(hWnd, L"Лабораторна робота №1", L"Про програму", MB_OK | MB_ICONINFORMATION);
            return 0;

        case IDM_EXIT:
            DestroyWindow(hWnd);
            return 0;
    }
    break;

case WM_PAINT:
    {
        PAINTSTRUCT ps{};
        HDC hdc = BeginPaint(hWnd, &ps);
        DrawTextLine(hdc, 30, 40, L"Лабораторна робота №1");
        DrawTextLine(hdc, 30, 85, L"Робота 1 - введений текст:");
        DrawTextLine(hdc, 30, 115, g_text);

        wchar_t numberBuffer[100]{};
        swprintf_s(numberBuffer, _countof(numberBuffer), L"Робота 2 - вибране число: %d", g_number);
        DrawTextLine(hdc, 30, 165, numberBuffer);
        EndPaint(hWnd, &ps);
        return 0;
    }
}
```

Після успішного завершення діалогу InvalidateRect ініціює повторне малювання. Текст виводиться через TextOutW, а числовий результат попередньо перетворюється на рядок функцією swprintf_s.

6. Модуль «Робота 1»

Заголовок module1.h містить тільки одну публічну функцію:

```
extern int Func_MOD1(HWND hwnd, wchar_t* buffer, int bufferSize);
```

Діалогова процедура є внутрішньою для модуля. При натисканні «Так» текст з Edit Control копіюється у буфер головного модуля.

```
static INT_PTR CALLBACK Work1DlgProc(HWND hDlg, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {
        case WM_INITDIALOG:
            return TRUE;

        case WM_COMMAND:
            switch (LOWORD(wParam))
            {
                case IDOK:
                    if (g_buffer != nullptr && g_bufferSize > 0)
                    {
                        GetDlgItemTextW(hDlg, IDC_EDIT_MOD1, g_buffer, g_bufferSize);
                        EndDialog(hDlg, 1);
                        return TRUE;
                    }
                    case IDCANCEL:
                        EndDialog(hDlg, 0);
                        return TRUE;
            }
            break;
    }
    return FALSE;
}

int Func_MOD1(HWND hwnd, wchar_t* buffer, int bufferSize)
{
    g_buffer = buffer;
    g_bufferSize = bufferSize;
    return static_cast<int>(DialogBoxW(
        GetModuleHandleW(nullptr),
        MAKEINTRESOURCEW(IDD_DIALOG_MOD1),
        hwnd,
        Work1DlgProc));
}
```

7. Модуль «Робота 2»

Зовнішній інтерфейс module2.h складається з однієї функції Func_MOD2. Модуль встановлює діапазон Scroll Bar від 1 до 100 та обробляє повідомлення WM_HSCROLL.

```
extern      int      Func_MOD2(HWND      hWnd,      int*      value);
static
{
    void      UpdateValue(HWND      hDlg)
    {
        HWND      hScroll      =      GetDlgItem(hDlg,      IDC_SCROLL_MOD2);
        SetScrollPos(hScroll,      SB_CTL,      g_value,      TRUE);
        SetDlgItemInt(hDlg,      IDC_VALUE_MOD2,      g_value,      FALSE);
    }

    case      WM_INITDIALOG:
        SetScrollRange(GetDlgItem(hDlg,      IDC_SCROLL_MOD2),      SB_CTL,      1,      100,      TRUE);
        UpdateValue(hDlg);
        return      TRUE;

    case      WM_HSCROLL:
        switch      (LOWORD(wParam))
        {
            case      SB_LINELEFT:      g_value--;      break;
            case      SB_LINERIGHT:      g_value++;      break;
            case      SB_PAGELEFT:      g_value      -=      10;      break;
            case      SB_PAGERIGHT:      g_value      +=      10;      break;
            case      SB_THUMBTRACK:
                g_value      =      HIWORD(wParam);      break;
        }
        if      (g_value      <      1)      g_value      =      1;
        if      (g_value      >      100)      g_value      =      100;
        UpdateValue(hDlg);
        return      TRUE;

    case      IDOK:
        if      (g_resultValue      !=      nullptr)      *g_resultValue      =      g_value;
        EndDialog(hDlg,      1);

    return TRUE;
}
```

Таким чином, стрілки змінюють значення на 1, сторінкові переходи - на 10, а перетягування повзунка безпосередньо задає позицію. Значення додатково обмежується межами 1-100.

8. Ресурси програми

У resource.h визначено ідентифікатори меню, команд і елементів керування. Головне меню знаходиться у Lab1.rc, а діалоги - в окремих module1.rc і module2.rc.

```
#define IDR_MAINMENU 101
#define IDM_WORK1 40001
#define IDM_WORK2 40002
#define IDM_EXIT 40003
#define IDM_ABOUT 40004
#define IDD_DIALOG_MOD1 201
#define IDC_EDIT_MOD1 1001
#define IDD_DIALOG_MOD2 202
#define IDC_SCROLL_MOD2 1002
#define IDC_VALUE_MOD2 1003

IDR_MAINMENU MENU
BEGIN
    POPUP "&Файл"
    {
        POPUP "&Робота"
        {
            MENUITEM "Робота 1", IDM_WORK1
            MENUITEM "Робота 2", IDM_WORK2
        }
        POPUP "&Довідка"
        {
            MENUITEM "Про програму", IDM_ABOUT
        }
    }
END

// module1.rc
EDITTEXT IDC_EDIT_MOD1, 10, 28, 210, 14, ES_AUTOHSCROLL
DEFPUSHBUTTON "Так", IDOK, 65, 60, 45, 15
PUSHBUTTON "Відміна", IDCANCEL, 120, 60, 50, 15

// module2.rc
LTEXT "Вибране число:", -1, 10, 12, 75, 12
LTEXT "1", IDC_VALUE_MOD2, 90, 12, 40, 12
SCROLLBAR IDC_SCROLL_MOD2, 10, 32, 220, 14, SBS_HORZ
DEFPUSHBUTTON "Так", IDOK, 68, 68, 45, 15
PUSHBUTTON "Відміна", IDCANCEL, 125, 68, 50, 15
```

Розміщення діалогів в окремих .rc-файлах дозволяє зберегти модульну організацію програми та не змішувати ресурси двох незалежних робіт.

9. Результати виконання - Робота 1

Після вибору пункту меню «Робота 1» відкривається модальне вікно з полем введення. Після підтвердження введений рядок відображається у головному вікні.

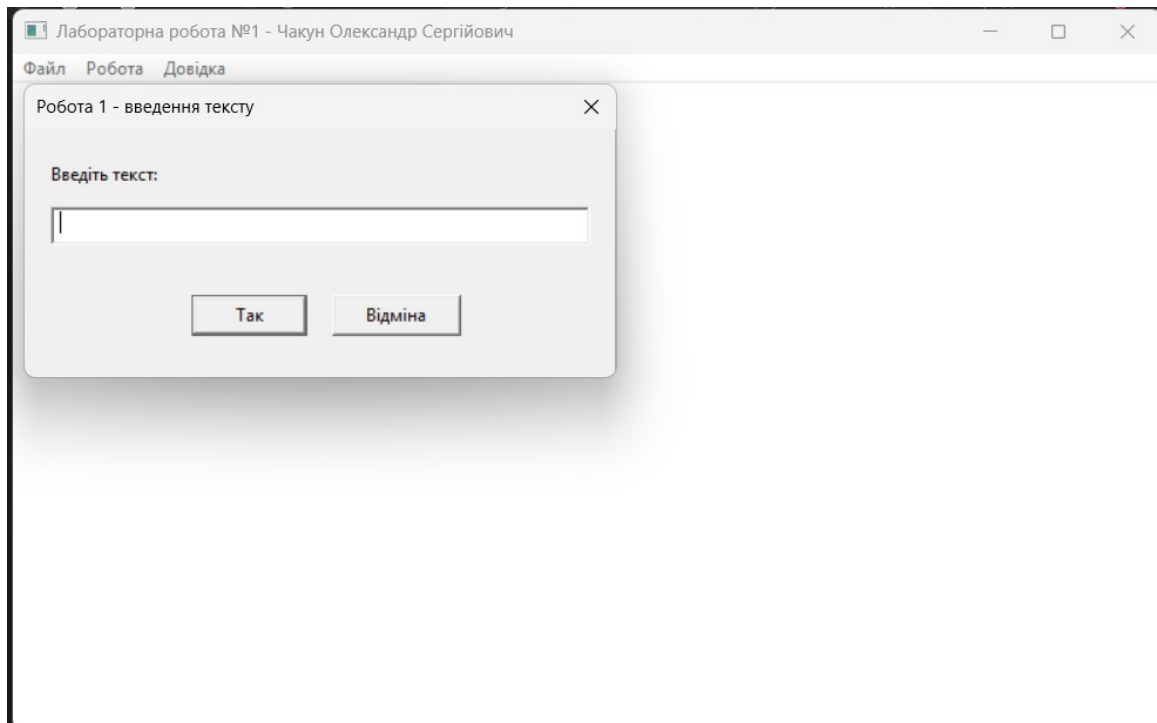
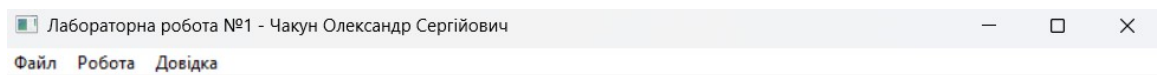


Рисунок 1 - Діалогове вікно «Робота 1»



Лабораторна робота №1

Робота 1 - введений текст:
ООП лабораторна 1

Робота 2 - вибране число: 1

Рисунок 2 - Виведення введеного тексту в головному вікні

10. Результати виконання - Робота 2

У другому діалозі користувач змінює значення горизонтального Scroll Bar. На прикладі вибрано число 50; після натискання «Так» воно відображається у головному вікні.

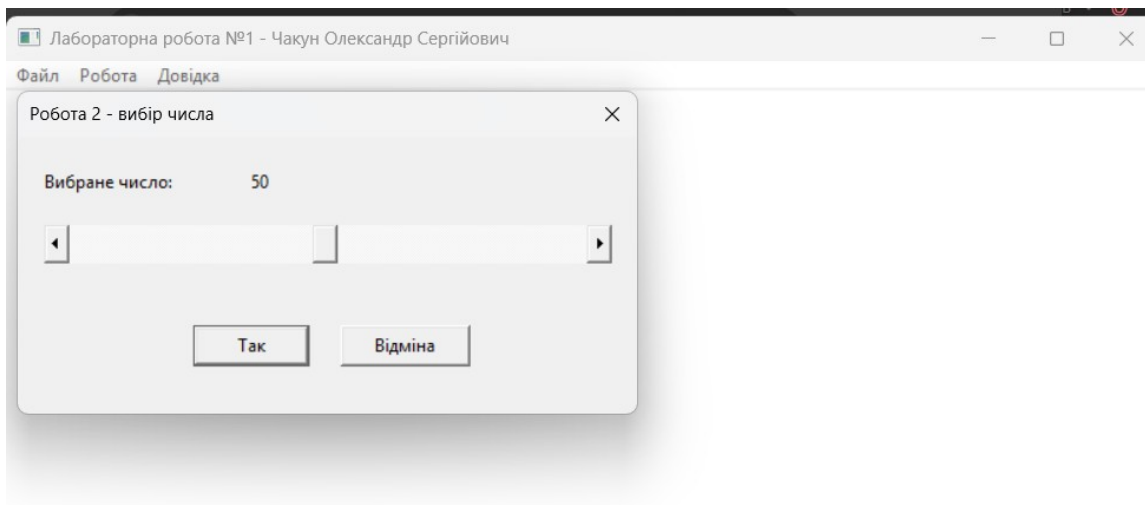
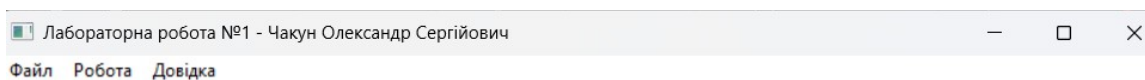


Рисунок 3 - Діалогове вікно «Робота 2»



Лабораторна робота №1

Робота 1 - введений текст:

ООП лабораторна 1

Робота 2 - вибране число: 50

Рисунок 4 - Результат вибору числа 50

11. Інформаційне вікно та висновки

Пункт меню «Довідка - Про програму» відкриває інформаційне вікно з назвою дисципліни, ПІБ виконавця, номером у списку та розрахованими варіантами.

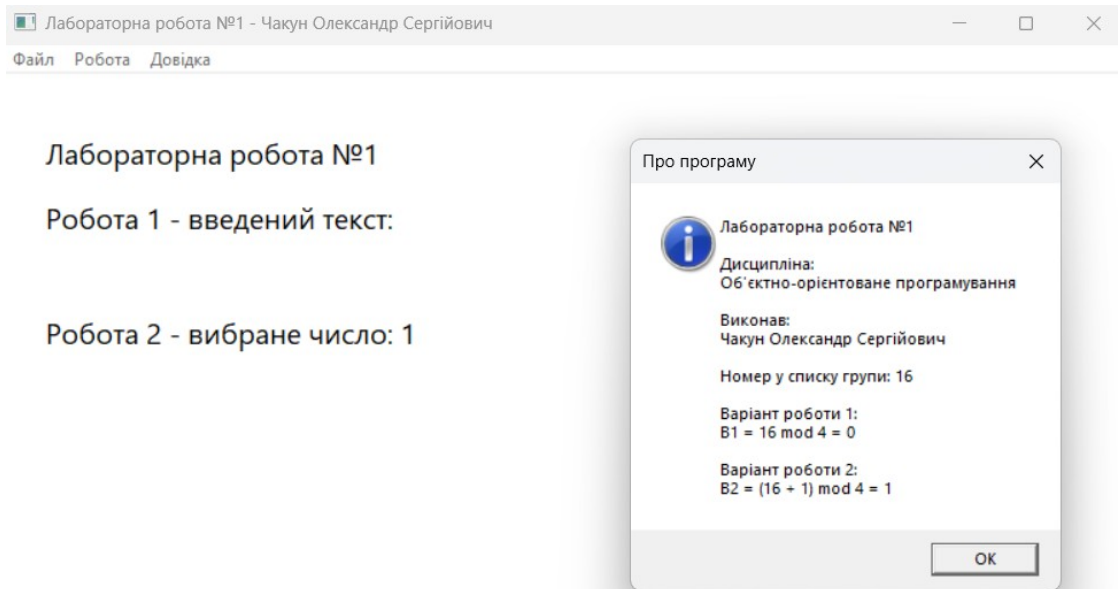


Рисунок 5 - Вікно «Про програму»

Висновки

У лабораторній роботі створено модульний Win32-застосунок мовою C++ з двома незалежними діалоговими модулями. Для варіанта $B1 = 0$ реалізовано введення тексту через Edit Control, а для варіанта $B2 = 1$ - вибір числа від 1 до 100 за допомогою горизонтального Scroll Bar.

Закріплено роботу з wWinMain, Callback-функціями, повідомленнями WM_COMMAND, WM_PAINT і WM_HSCROLL, ресурсними файлами, модальними діалогами та роздільною компіляцією. Публічний інтерфейс кожного модуля обмежено однією функцією, а внутрішні процедури приховано за допомогою static. Програма коректно виконує обидва індивідуальні завдання та відображає отримані результати.